

Apparel Recommendation

Date _____

Page _____

Case Study 8: Amazon Fashion Discovery Engine (Content based Recommendation)

V1: Problem Statement: Recommend similar apparel products in e-commerce using product descriptions and Images

Title has lot of info.
Price is also there.
Product description.

- Sponsored products related to this item. (Similar item)
- Customer who viewed this ~~at~~ item also viewed. (product recommendation).

Why care about product recommendation?

Estimated that 35% of revenue is due to product recommendation. (- on)

Internally Amazon uses:-

- 1) Content based recommendation. Uses Text and Image description to recommend.
- 2) Collaborative filtering based technique

$u_1 \rightarrow \underline{i_1}, i_2, \textcircled{i_3}$

$u_2 \rightarrow \underline{i_1}, \textcircled{i_3}, i_4$

Recommendation

$u_3 \rightarrow \underline{i_1}, \underline{i_3}$

$\hookrightarrow$ can be recommended

1) Recommendation technique will be used in this CS is Content based recom.

V2: Plan of Action

V3: Amazon product advertising API

API to download Amazon data in a policy compliant manner instead of scraping

V4: Data Folders and Paths

V5: Overview of the data and Terminology

```
data = pd.read_json('tops-fashion.json')
```

183138 — datapoints
Features — 19

Out of 19 only 6 features would be used.

asin - Amazon standard identification no.

V6: Data cleaning and understanding
: Missing data in various
features.

V7: Understand duplicate rows

28K datapoints are in pickle file $\rightarrow$ 28K apparel data.

calculating duplicating titles.
pandas dataframe object. dupli-
-cated

Images are present in dedupe
Each have ASIN, Amazon Standard

Identification Number.

V8: Remove duplicates: Part 1

1st preprocessing step

→ Split title, and if length is ≥ 4 then only keep it.

27,949 products remain.

→ Sorting products by title in DESC.

perform set difference in T_1 title and T_2 title.

If the difference is ≤ 2 words we will discard that title.

Do it in for loop.

for $i = 1$ to last line
for $j = i+1$ to last line

Comparison.

while loop can also be used. So 17593 data points are left.

Removed titles which differ only in the last few words

V9: Remove duplicates: Part 2

Some more duplicates but in different form.

T_1
T_2
$\vdots$
T_n

$(i, j) \quad i: 1 \rightarrow n$
 $j: 1 \rightarrow n$

every pair of (i, j)

→ find out how many words they differ in?

$n = 17K$, nC_2 combinations or pairs
 $O(n^2)$ complexity.

$$\frac{17593 * (17593 - 1)}{2} = 154MM \text{ Comparisons.}$$

We are doing a Brute Force algorithm.

Better algorithm's such as Inverted Indexes. → To compute no. of differing words against other titles. → 16K apparel data.

V10: Text Pre-processing: Tokenization and Stop word Removal

Remove stop words in title.

V11: Stemming

```
nlTK.Stem.porter import *
Stemmer = PorterStemmer()
```

Stemming $\rightarrow$ Finds Root of word.

2:45 mins

V12 - Text based product Similarity Converting text to an n-D vector Bag of words

16K product titles.

only Titles are used. They are short and Informative.

We have de-duped and removed stop words.

$T_1, T_2, T_3, \dots, T_i, \dots, T_n$ $n \sim 16k$

Euclidean distance to find similar points.

Product $\rightarrow$ Title $\rightarrow$ d-dim point
 $(P_1, P_2 \dots P_n) (T_1, T_2 \dots T_n)$

We can use euclidean distance to find similar products.

We want to represent Text which is title as a d-dim vector/point which is array of size d.

Bag of words (BoW)

$T_1, T_2, T_3, \dots, T_n \rightarrow (\text{Text})$

1) Set of words which are present in any of the titles.

$S = \text{Set of all the words present in } T_1, T_2, \dots, T_n$

2) $W_1, W_2, W_3, \dots, W_d$ is =

12,609.

$W_1 \ W_2 \ W_3$

$P_i \rightarrow T_i$

0	1	2					
---	---	---	--	--	--	--	--

 -

No. of times
 W_j is present
in title T_i

d-dim vector
array of
size d.

eg $\rightarrow$ zago tshirt long sleeve

tshirt XL

$\rightarrow T_i$

L	XL	tshirt	short	zago	long	slee
0	1	2	0	1	1	0

 1

BoW

- Simple
- Count based
- No sequence info is stored.

V13: Code for bag of words based product similarity

Code walkthrough.

Sparse vectors $\rightarrow$ mostly zeroes.

Helps reduce memory issues by saving a lot of space.

Function pairwise_distances
to calculate distance b/w points
in sklearn. With this we
can calculate distance b/w
nearest neighbours.

Euclidean distance is calculated
using $\sqrt{\text{sum of sq. of distance}}$

We can show euclidean ^{difference}
similarity with the query
image.

V14: TF-IDF: Featurizing text based on word importance

First approach Text Bow $\rightarrow$ n-Dim Vector

2nd approach TFIDF

TF - Term Frequency.

IDF - Inverse Document Frequency.

T_1 :
 T_2 :
 T_3 :
 $\vdots$
 T_n :

} Document
Corpus

classmate
Date _____
Page _____

for a given word → for a given text

$$\textcircled{1} \text{TF}(w_j, T_i) = \frac{\# \text{ times } w_j \text{ occurs in } T_i}{\# \text{ words in } T_i}$$

If Text T_i has k words. If w_j occurs twice.

$$\text{Then } \text{TF}(w_j, T_i) = \frac{2}{k}$$

TF is large if w_j occurs multiple times in T_i .

* Term Frequency → occurrence no.

$$\textcircled{2} \text{IDF}(w_j, D) = \ln \left\{ \frac{\# \text{ of documents}}{\# \text{ docs in } D \text{ contain the word } w_j} \right\}$$

word Document Corpus natural log.

Let $D = 16k$ and $12k$ are tshirt

$$\text{Then } \text{IDF}(\text{tshirt}, D) = \ln \left(\frac{16k}{12k} \right) = 0.1$$

$$\text{IDF}(\text{tiger}, D) = \ln \left(\frac{16k}{200} \right) = 2.20$$

$$\text{IDF}(\text{tiger}, D) > \text{IDF}(\text{tshirt}, D)$$

IDF measures how rare a word is. It increases if the word is rare.

(*)

TFIDF

$T_i \rightarrow W_1, W_2, W_3, W_4$

$\rightarrow$ **BOW**
(word freq.)

0	1	0	1	0	1	0	1
W_1		W_2		W_3		W_4	

12609

$\rightarrow$ **TFIDF** = $TF(W_i, T_i) * IDF(W_i, D)$
(TFIDF freq.)

$D = (T_1, T_2, \dots, T_n)$

W_1		W_2		W_3		W_4	

12609

in D
that

If W_i occurs ~~to~~ very much in corpus then $TF * IDF(W_i)$
 if $TF \uparrow$ and $IDF \downarrow$
 if it's rare $TF \downarrow$ and $IDF \uparrow$.

code

V15: TF-IDF based product similarity

TfidfVectorizer — sklearn



Scipy Sparse matrix

TFIDF: Builds vector according to word importance.V16: Code for ~~TAM~~ IDF based product similarity

Observation → words do not repeat often.

$$\rightarrow TF(W_i, T_1) = \frac{1}{10}$$

 $(T_1: W_1 W_2 W_3 W_4 W_5 W_6 W_7 W_8 W_9 W_{10})$
 $(T_2: W_1 W_3 W_6 W_8)$

$$\rightarrow TF(W_i, T_2) = \frac{1}{4}$$

TF * IDF

TF ↑ for shorter titles.

TF-IDF for search engines in large documents.

$$TF(w_j, T_i) = \frac{\text{freq. of } w_j \text{ in } T_i}{\# \text{ words in } T_i}$$

→ Drop TF & use IDF only.
Experiment.

def idf

V17: Text Semantics based product similarity: word2Vec (featurizing text based on semantic similarity)

eg → Zebra stripes

 tiger leopard

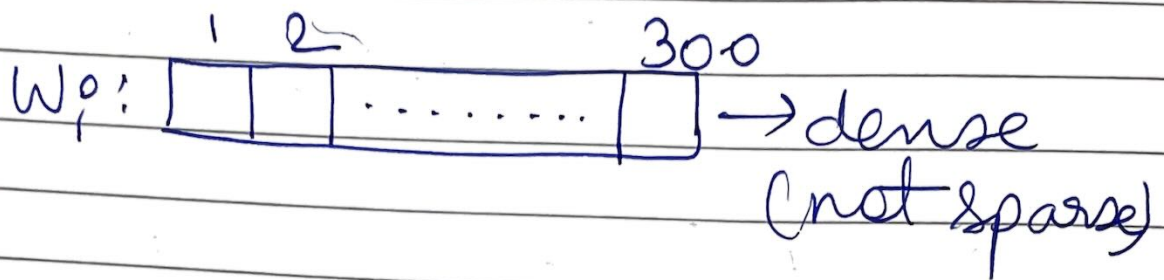
BoW or TFIDF → do not account for semantics.

Title → Vector (d-dim array) →

Euclidean distance for similarity.

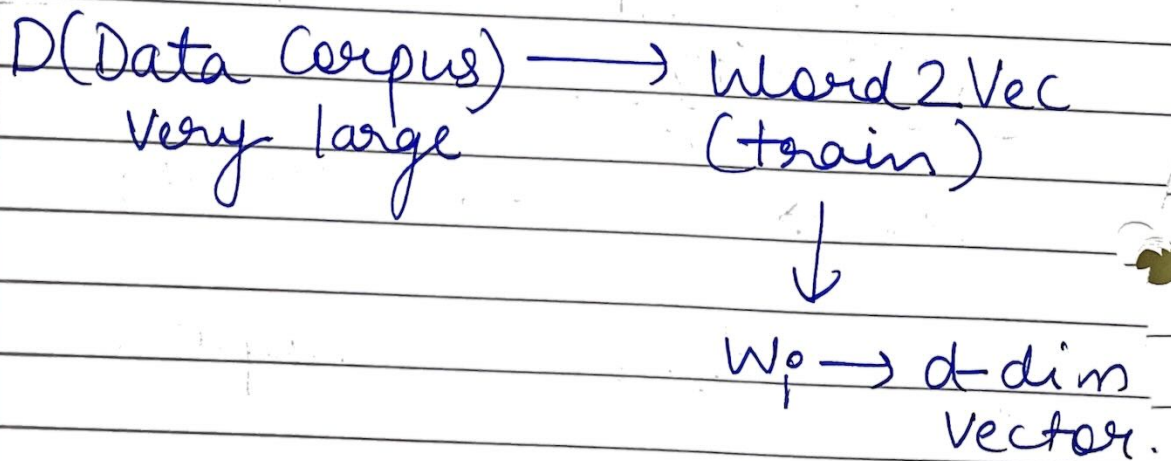
Word2Vec (2013) @ Google inventio

Word2Vec



BOW & TFIDF $\rightarrow$ very sparse vectors.

most of the cells are non-sparse vectors.



geometric intuition of word2vec

W2V: Super-powerful
semantic relationships
in the d-dim vectors

n. Q) How does W2V algo learn all of this from D?

(Word-Context) is used.

$W_1, W_2, W_3, W_4, W_5, W_5$
 (Context)

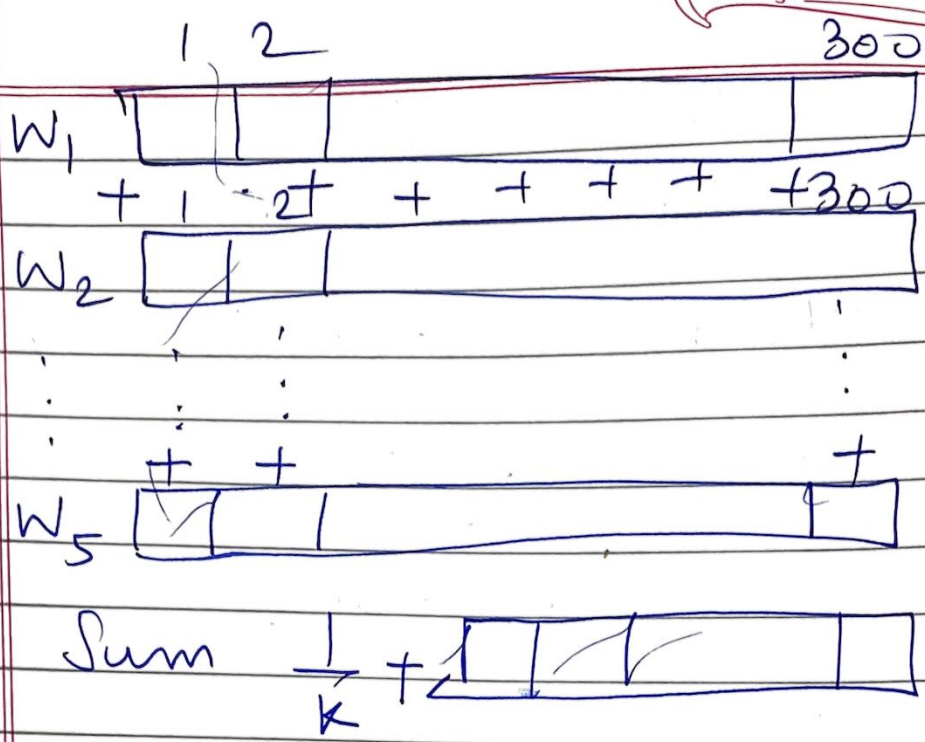
exact algo :- beyond the scope

$D(\text{large corpus}) \xrightarrow{W2V} W_i : \rightarrow$
 d-dim dense
 Vector

$(T_1, T_2 \dots T_n) \rightarrow \text{Small}$.

We will use Google News built 300 dim W2V.

We care about building W2V for our titles using Google's 300 dim W2V model.



$$W_2 V(T_i)$$

$$\text{Avg } W_2 V(T_i)$$

We sum W_1 of T_1, W_1 of $T_2 \dots T_5$ and so on.

Similarly W_2 of T_1, T_2 of T_2 and W_2 of T_5

and so on

and we take an average of it.

So, it is called Avg $W_2 V$.

Own dictionary from google news sampled data and is available as a pickle on g-drive

heatmap gives euclidean distance b/w value on x-axis and y-axis.

Lower the euclidean distance more similar are the objects on respective axes.

$$D = \{T_1, T_2, \dots, T_n\}$$

T_1 W_1 W_2 W_3 W_4 W_5 ①
300

W_1 - - - - - * TF-IDF
 W_2 - - - - - of (W_i, T_i, D)

W_5

 * T F I D F J
 (W_5, T_i, D_i)

(2) SUM: (3) Add to get N

③④ Multiply $\frac{1}{2}$ with ②

we use word-importance in TFIDFW2V

V22: Code for weighted similarity
weights show our preference.

V23: Building a real world Solution
we've used.

- 1) BoW
- 2) TFIDF
- 3) Avg W2V
- 4) TFIDFW2V
- 5) Weighted-Brand-color

Companies use multiple result.

often at Industry-scale projects
all of the above is considered
so we apply business rule.

Business rules could be :-

- 1) Do not show products of same brand name more than twice
- 2) Show the final list after combining.

Colors - $C_1, C_2, \dots, C_k$ k colors

$P_i \rightarrow C_i: \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} \leftarrow \text{color vector}$

hot encoding

$P_i: \begin{bmatrix} \text{Title} & \text{Brand} & \text{color} \end{bmatrix}$
 300 m k } Product vector

Concatenated

euclidean distance (P_i, P_j)

ty.

Let's assume:-
 prefer showing our customers product of the same brand.

modify euclidean dist $\rightarrow$ weighted euclidean distance

$$\left. \begin{array}{l} W_T * \text{Title} \\ W_B * \text{Brand} \\ W_C * \text{color} \end{array} \right\} \div \text{Sum}(W_T, W_B, W_C)$$

If $W_T = 1, W_B = 5, W_C = 1$.
 you end up preferring products of same brand.

A/B Testing / Live Testing / Bucket Testing

V24: Deep learning based visual product similarity: ConvNets:
How to featurize an image:
edges, shapes, parts

Text (Title), Brand color

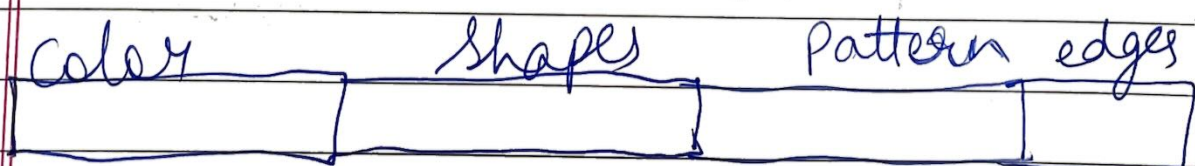
BoW, tfidf, one-hot
 W2V

For every product u_i there is an image $u_{i,1}$.

How to featurize an image into an n -Dim Vector?

In an image there are :-

- edges
- shapes
- color
- pattern (zebra stripes)
(leopard print)



euclidean distance for similarity.

Till 2012 everything was done manually or like this.

But from 2012, we can use deep learning we can automatically convert image to n-D vector using CNN. [-call]

ConvNets

Alex Nets

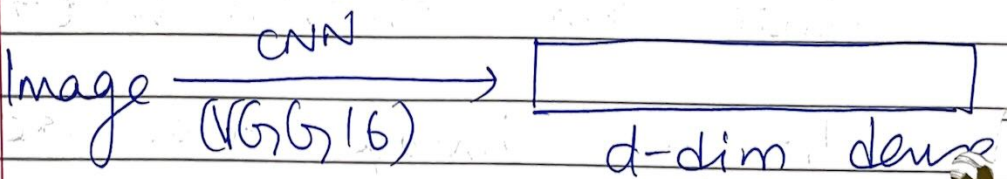
VGG16

VGG19

GoogLeNet

ResNet

} CNN models.



are similar, euc-dist small.

y_1, y_2

$\downarrow \downarrow$

v_1, v_2

euclidean distance

(v_1, v_2)

$\downarrow$
Small.

V25: Using Keras + Tensorflow to extract features

VGG-16 (developed @ Oxford Univ)

V26: Visual similarity based product similarity

• numpy image features are saved in gdrive.

V27: Measuring goodness of our solutions: A/B Testing

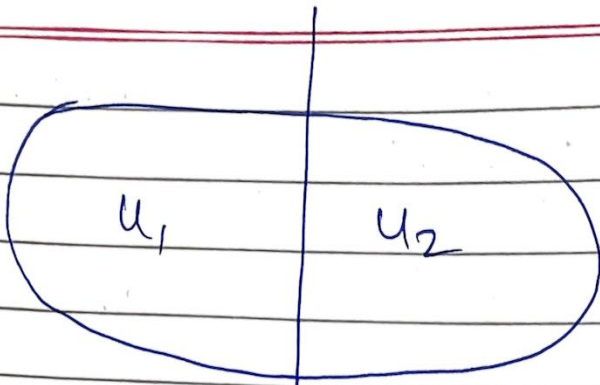
P_Q → Sol. 1 → ☐ ☐ ☐ ☐ ☐

P_Q → Sol. 2 → ☐ ☐ ☐ ☐ ☐

Solution :- Pre-processing.
Algorithm.
Business rules.

Q How do you evaluate which solution is better?

A → Statistical Testing called A/B Testing
Bucket "



users on
Amazon.com

non-overlapping groups u_1 & u_2
split randomly.

For u_1 , we show sol_1 .

For u_2 we show sol_2 .

For sol_1 , we measure sales.

For sol_2 , " " "

If sales on u_1 (using sol_1)
is $>$ sales on u_2 (using sol_2).

We say customers like
 sol_1 than sol_2 .

Assignment M-24 (Apparel Recommendation).

Following features have been used:

1) Text $\rightarrow$ Bag of Words, TF-IDF, Aug W2V, TFIDF-W2V, IDF-W2V

2) Weighted Text features, Brand, Color

3) Image $\rightarrow$ Image features using (VGG 16)

Take these best features

Give $\rightarrow$ Weight $\rightarrow$ Give Weight B	Give W_c	CNN	16
IDF W2V	One-hot	One-hot	VGG/16
Text	Brand	Color	Image

Give W_i
25000
D
features

Try to build $\downarrow$

Weighted euclidean-distance based similarity